

Concrete Architecture Evaluation Report (T1)

Delivery Date: Apr 25, 2026

Team Members: Rukiye Sedef Keskin 2210356150

Supervisor: Prof. Ayça Kolukisa

(<https://avesis.hacettepe.edu.tr/atarhan>)

1. OSS PRODUCT OVERVIEW

Table 1. Description of the OSS product

Name of OSS Product:	Thymeleaf
Description of OSS product:	Thymeleaf is a modern server-side Java template engine developed for both web and standalone environments. Its main purpose is to introduce 'natural templates' into the development process.
URL of OSS product:	https://github.com/thymeleaf/thymeleaf
Size of OSS product (KLOC):	110KLOC

2. TOOLS USED FOR ANALYSIS

Table 2. The list of tools used for analysis

Tool Name	Purpose of Use	URL
SciTool Understand	To obtain maintainability metrics, To create dependency graphs	https://scitools.com/
C&K Extended	To obtain maintainability metrics	https://gromit.iia.pwr.wroc.pl/p_inf/ckjm/

3. QQM TREE FOR EVALUATING OSS

Table 3. The goal, questions, and metrics to evaluate OSS maintainability. Appendix A includes visualization of QQM Tree.

Goal: Evaluate the maintainability of Thymeleaf source code		
List of questions to answer while evaluating the goal:		List of metrics used to answer the question:
Q(1):	What is the coupling level between classes?	CBO – 7 (median) RFC – 13 (median) MPC – 6 (median) CF – %21 (CF>0)
Q(2):	What is the complexity level of the code?	WMC – 5 (median) CC – 12.33 (median) NNL – 8 (max) H_V – 2369 (median) H_D – 31.88 (median) H_E – 74.25 (median) H_B – 0.79 (median)
Q(3):	Is class cohesion sufficient?	LCOM – 3 (median) NOM – 4 (median)

Q(4)	Is the inheritance hierarchy appropriate?	DIT – 6 (max) NOC – 14 (max) - %18 (NOC >0) AIF – 0.51 (avg) MIF – 123 (max) - %31.6 (MIF > 0)
Q(5)	What is the code size and volume?	LOC – 69 (median) SLOC – 37 (median) H_N – 386 (median) H_n – 72 (median) H_N1 – 231 (median) H_N2 – 147 (median) H_n1 – 21.89 (avg) H_n2 – 50 (median)
Q(6)	What is the encapsulation quality?	AHF – 72 (median) MHF – 0.16 (avg) PF – 0.133 (avg) - %25 (PF>0)
Q(7)	What is the overall maintainability level?	MI – 73.27 (median)

4. DESCRIPTION OF SOFTWARE METRICS

Table 4. List of software metrics used in analysis

Metric Name	Type	Source	Formula (or description) by Source
CBO	C&K	CKJM Tool	Coupling Between Objects measures the number of distinct classes to which a given class is directly coupled. The widely accepted threshold in the literature is ≤ 10 (efferent + afferent coupling).
RFC	C&K	CKJM Tool	Response For a Class represents the total number of methods that can potentially be executed in response to a message received by a class.
MPC	Halstead	SciTool Understand	Message Passing Coupling measures the number of method invocations (messages) sent by a class to other classes.
CF	MOOD	CKJM Tool	Coupling Factor represents the ratio of actual couplings between class pairs to the total number of possible couplings in the system. Computed as CBM count (proxy; not a true 0–1 ratio).
WMC	C&K	CKJM Tool	Weighted Methods per Class is the sum of the cyclomatic complexities of all methods declared within a class.
CC	Other...	CKJM Tool	Cyclomatic Complexity measures the number of linearly independent execution paths through a method. Reported here as the average cyclomatic complexity per method within a class.
NNL	Other...	SciTool Understand	Maximum Nesting Level denotes the deepest level of nested control structures (e.g., if, for, while) within a class. The generally recommended threshold is $\leq 3-4$ levels.
H_V	Halstead	<i>javalang</i>	Halstead Volume represents the total number of bits required to encode a program. Computed as: $V = N \times \log_2(n)$.
H_D	Halstead	<i>javalang</i>	Halstead Difficulty serves as an indicator of the mental effort required to comprehend the code. Computed as: $D = (n_1 / 2) \times (N_2 / n_2)$.
H_E	Halstead	<i>javalang</i>	Halstead Effort quantifies the total mental workload required to write and understand a module. Computed as: $E = V \times D$.
H_B	Halstead	<i>javalang</i>	Halstead Bugs estimates the expected number of errors delivered in a module. Computed as: $B = V / 3000$.
LCOM	C&K	CKJM Tool	Lack of Cohesion in Methods measures the degree to which the methods of a class do not share common instance variables. Higher values indicate lower cohesion.
NOM	Halstead	SciTool Understand	Number of Methods denotes the total count of methods (public and private) declared within a class.
DIT	C&K	SciTool Understand	Depth of Inheritance Tree measures the length of the longest path from a class to the root of the inheritance hierarchy.
NOC	C&K	SciTool Understand	Number of Children denotes the count of immediate subclasses of a given class.

AIF	MOOD	SciTool Understand	Attribute Inheritance Factor measures the proportion of a class's attributes that are inherited rather than newly defined. Computed as: $AIF = \text{inherited attributes} / (\text{inherited} + \text{defined attributes})$; range: 0–1.
MIF	MOOD	CKJM Tool	Method Inheritance Factor represents the ratio of inherited methods to the total number of methods available in a class. Computed as a proxy count of inherited methods (true MIF requires ratio normalization); range: 0–1.
LOC	Other...	SciTool Understand	Lines of Code denotes the total number of lines in a source file, including comments and blank lines.
SLOC	Halstead	SciTool Understand	Source Lines of Code represents the number of executable code lines, excluding comments and blank lines.
H_N	Halstead	<i>javalang</i>	Halstead Length is the total number of operator and operand occurrences in a module. Computed as: $N = N_1 + N_2$.
H_n	Halstead	<i>javalang</i>	Halstead Vocabulary reflects the diversity of unique tokens used in a class. Computed as: $n = n_1 + n_2$.
H_N1	Halstead	<i>javalang</i>	Total Operator Occurrences — the total count of operator tokens appearing in the source code.
H_N2	Halstead	<i>javalang</i>	Total Operand Occurrences — the total count of operand tokens appearing in the source code.
H_n1	Halstead	<i>javalang</i>	Distinct Operators — the number of unique operator types used in the source code.
H_n2	Halstead	<i>javalang</i>	Distinct Operands — the number of unique operand types used in the source code.
AHF	MOOD	CKJM	Attribute Hiding Factor measures the proportion of a class's fields that are concealed from external access. Computed as the ratio of private and protected attributes to total attributes. Reported as a proxy count (true AHF requires system-level normalization).
MHF	MOOD	SciTool Understand	Method Hiding Factor measures the proportion of a class's methods that are inaccessible to external clients. Computed as: $MHF = (\text{private} + \text{protected methods}) / \text{total methods}$; range: 0–1.
PF	MOOD	<i>javalang</i>	Polymorphism Factor represents the ratio of overriding methods to the total number of methods eligible for overriding within a class. Computed as: $PF = @Override \text{ methods} / \text{total methods}$; range: 0–1.
MI	Other...	<i>javalang</i>	Maintainability Index is a composite metric estimating the ease of maintaining source code. Computed as: $MI = \max(0, (171 - 5.2 \times \ln(H_V) - 0.23 \times CC - 16.2 \times \ln(LOC)) \times 100 / 171)$.

5. EVALUATING OSS

5.1. Answer to “What is the coupling level between classes?”

The coupling level of Thymeleaf was assessed through four metrics:

- **CBO = 7 (median):** Falls below the critical threshold of 10, indicating acceptable inter-class coupling. Classes with CBO > 10 in the upper tail should be reviewed as refactoring candidates.
- **RFC = 13 (median):** Reflects moderate external method invocations per class. This is a satisfactory value from a testability standpoint, as fewer dependencies need to be mocked during unit testing.
- **MPC = 6 (median):** Consistent with the CBO value — classes communicate with approximately 7 distinct classes via 6 method calls on average, indicating a proportional and controlled coupling profile.
- **CF = 21%:** Only 21% of all possible class pairs share a direct coupling relationship, which is well below the theoretical maximum of 100%, signaling a modular and loosely coupled architecture.

- **Overall**, coupling in Thymeleaf is controlled and within acceptable bounds. However, since all values are median-based, outliers in the upper distribution of CBO and RFC should be individually examined to identify structurally problematic classes.

5.2. Answer to “What is the complexity level of the code?”

The complexity of Thymeleaf was assessed through seven metrics:

- **WMC = 5 (median)**: Indicates low overall complexity per class, suggesting that classes are focused and do not suffer from excessive growth. The widely used "God Class" threshold of $WMC > 20$ is well above this value, which is a reassuring result.
- **CC = 12.33 (median)**: Sits just above the moderate complexity threshold, implying that a typical method requires at least 12 test scenarios for adequate coverage. Methods with $CC > 20$ in the upper tail are high-priority refactoring candidates.
- **NNL = 8 (maximum)**: A nesting depth of up to 8 levels signals a readability risk in certain methods. Deep nesting complicates both comprehension and testing; applying guard clauses or the extract method pattern is recommended.
- **H_V = 2.369 (median)**: Reflects a moderate volume per method — neither too small nor overly inflated — suggesting a balanced level of implementation detail.
- **H_D = 31.88 (median)**: While no strict threshold exists, this value is noteworthy for a template engine such as Thymeleaf and deserves monitoring.
- **H_E = 74.25 (median)**: Represents a moderate cognitive effort for developers; adaptation to the codebase is expected to take reasonable but not excessive time.
- **H_B = 0.79 (median)**: Fewer than one estimated bug per class on average, which is a positive indicator of overall code reliability.
- **Overall**, class-level complexity (WMC) is well under control; however, method-level cyclomatic complexity ($CC = 12.33$) and maximum nesting depth ($NNL = 8$) are the primary risk factors. Halstead metrics suggest a moderate cognitive load overall, with H_B remaining low — a favorable signal for code quality. Priority action: methods with $CC > 15$ and blocks with $NNL > 4$ should be scheduled for refactoring.

5.3. Answer to “Is class cohesion sufficient?”

The cohesion level of Thymeleaf was assessed through two metrics:

- **LCOM = 3 (median)**: Sits at the upper boundary of the acceptable range, suggesting that methods within a typical class do not fully share the class's data fields. This implies that some classes may encapsulate two or more loosely related responsibilities under a single unit. Classes with $LCOM > 6$ in the upper tail are priority candidates for review in terms of Single Responsibility Principle (SRP) violations.
- **NOM = 4 (median)**: Indicates a compact and focused class structure. The commonly referenced risk threshold of $NOM > 20$ is far above this value, confirming that classes do not suffer from an excessive method count or overloaded responsibilities.
- **Overall**, cohesion in Thymeleaf is neither insufficient nor strongly satisfactory. While the low NOM value confirms that classes remain lean, the median LCOM of 3 suggests that responsibility is not fully shared across methods within a class. Priority action: classes with $LCOM > 4$ should be reviewed for SRP compliance and, where necessary, decomposed into smaller, more focused units.

5.4. Answer to “Is the inheritance hierarchy appropriate?”

The inheritance structure of Thymeleaf was assessed through four metrics:

- **DIT = 6 (maximum):** Exceeds the commonly accepted depth threshold, indicating that the deepest inheritance chains introduce fragility and increased cognitive load. Since this is a maximum value, it reflects only the most extreme cases rather than the overall system.
- **NOC = 14 (max) | 18% (NOC > 0):** The maximum of 14 direct subclasses suggests that at least one parent class carries excessive responsibility and may be vulnerable to change. However, only 18% of all classes have any subclasses, indicating a selective and disciplined use of inheritance throughout the system.
- **AIF = 0.51 (average):** Approximately half of each class's attributes are inherited from parent classes, reflecting a balanced code reuse profile — neither fully independent nor overly dependent on ancestor classes.
- **MIF = 123 (max) | 31.6% (MIF > 0):** A maximum of 123 inherited methods signals that certain classes act as thin shells with minimal behavior of their own, which becomes a higher risk when combined with DIT = 6. Notably, 68.4% of classes inherit no methods at all, suggesting a composition-favoring design consistent with modern OOP practices.
- **Overall,** the inheritance hierarchy in Thymeleaf is largely controlled and balanced. The low proportion of subclassed (18%) and method-inheriting (31.6%) classes confirms a preference for composition over inheritance. Nevertheless, the extreme values of DIT and MIF indicate structurally problematic classes at the tail of the distribution. Priority action: inheritance chains with DIT > 4 and classes with exceptionally high MIF values should be re-evaluated in terms of favoring composition over inheritance.

5.5. Answer to “What is the code size and volume?”

The size and volume of Thymeleaf were assessed through eight metrics:

- **LOC = 69 (median):** Indicates a compact class structure, well below the 200–300 line threshold commonly associated with bloated classes in the literature.
- **SLOC = 37 (median):** The LOC/SLOC ratio reveals that approximately 46% of the codebase consists of comments and blank lines, reflecting a healthy balance between executable code and documentation.
- **H_N = 386 (median):** Represents a moderate program length. Combined with the median LOC of 69, this implies roughly 5–6 tokens per line on average, suggesting that overly complex expressions are avoided.
- **H_n = 72 (median):** A typical Thymeleaf class uses 72 distinct operator-operand combinations, representing an acceptable level of vocabulary diversity without indicating unnecessary complexity.
- **H_N1 = 231 (median):** Reflects a moderate level of operator repetition, indicating that control flow and operational density within classes remain at a manageable level.
- **H_N2 = 147 (median):** The H_N1/H_N2 ratio of approximately 1.57 suggests that operator usage outweighs operand usage, meaning the code is more focused on operations and control flow than on variable declarations — an expected characteristic for a template engine architecture.
- **H_n1 = 21.89 (median):** An average of 22 distinct operators per class indicates a balanced use of the language's available constructs, consistent with idiomatic Java development.
- **H_n2 = 50 (median):** Fifty unique operands per class confirms that classes operate within a moderate data universe — neither too small to be trivial nor too large to be unmanageable.
- **Overall,** all size and volume metrics present a positive picture for Thymeleaf. The median of 69 LOC and 37 SLOC confirms that the codebase is composed of small, focused classes free from bloat. Halstead metrics further support this interpretation, collectively indicating that developers have adopted a simple, readable, and disciplined coding style. For an open-source project of this scale (1,901 classes), these values establish a strong foundation for long-term maintainability.

5.6. Answer to “What is the encapsulation quality?”

The encapsulation quality of Thymeleaf was assessed through three metrics:

- **AHF = 72 (median):** Approximately 72% of class attributes are hidden from external access, placing the system near the upper boundary of the moderate-to-good range. The remaining 28% of publicly or package-privately accessible attributes represent a potential area for improvement, likely attributable to directly exposed fields rather than proper accessor methods.
- **MHF = 0.16 (average):** Only 16% of methods are defined as private or protected, leaving 84% of the method surface publicly accessible. This is the most critical finding in this section. While a large public API surface is architecturally justified for a framework such as Thymeleaf, internal utility methods that are not part of the public API should be reviewed for stricter access control.
- **PF = 0.133 (average) | 25% (PF > 0):** Polymorphism is used modestly — classes override approximately 13% of inherited methods on average, and only 25% of all classes exhibit any polymorphic behavior. This is consistent with the findings in Section 5.4, confirming that the system favors composition and direct implementation over inheritance-based polymorphism. While this is a defensible choice for a template engine, broader use of polymorphism could enhance extensibility.
- **Overall,** encapsulation quality in Thymeleaf presents a mixed picture. While AHF indicates a reasonable level of data hiding, the low MHF of 0.16 is the most notable concern — the vast majority of methods are externally accessible. Although the framework's wide API surface partially justifies this, restricting internal utility methods to private or protected access would meaningfully strengthen encapsulation. Priority action: access modifiers of non-public-facing internal methods should be audited and restricted where appropriate.

5.7. Answer to “What is the overall maintainability level?”

The overall maintainability of Thymeleaf was assessed through a single composite metric:

- **MI = 73.27 (median):** Places Thymeleaf in the **good maintainability** band, a notable result for a codebase of 1,901 classes. This confirms that the project upholds a consistent standard of code quality overall. Targeted improvements — particularly reducing cyclomatic complexity (CC) and increasing method hiding (MHF) — could realistically elevate the score into the **excellent** band ($MI \geq 85$).
- **Overall,** Thymeleaf demonstrates a solid maintainability profile for an open-source project of its scale. The MI score reflects the cumulative effect of the controlled coupling, moderate complexity, and compact class size observed in previous sections. Addressing the identified risk factors in CC and MHF represents the most direct path toward further improvement.

6. DEPENDENCY GRAPH(S)

A **dependency matrix (Appendix B)** is a tabular representation of inter-package relationships in which each cell indicates the number of references from one package to another, enabling quantitative comparison of coupling intensity across the system. A **dependency graph (Appendix C)**, by contrast, is a directed visual representation of the same relationships, where nodes represent packages and edges indicate dependencies with explicit directionality.

The two representations are complementary: the matrix provides quantitative breadth, while the graph reveals directionality, circular dependencies, and the architectural role of individual components — none of which are reliably detectable from tabular data alone. Both are therefore

included to enable a complete and accurate structural assessment.

6.1 Findings from the Dependency Matrix

The dependency matrix reveals the following key findings regarding Thymeleaf's inter-package coupling:

- **util — highest fan-out (16 non-zero entries):** The utility package exhibits the most extensive outgoing dependency profile in the system, connecting to virtually every other package. This pattern is architecturally concerning, as utility packages are conventionally expected to have minimal or no outgoing dependencies.
- **model → (target): 485 (critical intensity):** The highest single dependency value in the entire matrix, falling in the 351+ range. This indicates an extreme level of coupling between the model package and one of its dependencies, representing a significant fragility risk.
- **util → standard: 322 – | util → engine: 319:** Two additional extreme-intensity dependencies further reinforce the structural concern around the util package.
- **templatemode — 12 non-zero entries:** The second most connected package in the matrix, exhibiting a broad dependency surface across the system.
- **exceptions — ambiguous profile:** With 6 non-zero entries and values up to 212, the matrix appeared to suggest that the exceptions package has significant outgoing dependencies — an unexpected and concerning finding for a package of this type. This finding required further verification through the dependency graph.
- **Leaf packages confirmed:** `preprocessor`, `postprocessor`, `messengeresolver`, and `linkbuilder` each exhibit only 2–3 non-zero entries, confirming their role as well-isolated, low-dependency components.

6.2 Findings from the Dependency Graphs

The dependency graphs provided both confirmation and correction of the matrix findings, while revealing additional structural risks invisible in tabular form:

- **Circular dependencies identified (red edges):** The graph reveals multiple bidirectional dependency cycles, the most critical of which are `engine ↔ util` (186/35), `engine ↔ model` (485/7), `engine ↔ cache` (16/42), and `engine ↔ context` (34/108). These cycles introduce fragility, complicate testing, and impede independent module evolution. The matrix contained the underlying values but could not expose the cyclic nature of these relationships.
- **util is a fan-in hub, not a fan-out risk:** Graph 20 (Util — Code Level) reveals that util itself depends on virtually nothing — only `exceptions` with a value of 3. The high values observed in the matrix reflected incoming dependencies from 15+ packages, not outgoing ones. This is a significant correction to the matrix-based interpretation: util functions as a shared utility sink, which is architecturally appropriate, though the volume of incoming dependencies makes it a high-impact change surface.
- **exceptions is a pure sink:** Graph 3 (Exceptions — Code Level) confirms that the exceptions package has no outgoing dependencies whatsoever. All values observed in the matrix represented incoming references from packages such as `engine`, `expression`, `processor`, `standard`, and others. This is the correct architectural behavior for an exception package.
- **engine confirmed as central hub:** The L3 and L4 graphs consistently position `engine` at the structural center of the system, with both the highest number of incoming and outgoing connections. Its

involvement in multiple circular dependencies makes it the highest-priority refactoring target in the codebase.

- **templatemode** — **unexpectedly broad fan-in**: Graph 12 reveals that **templatemode** receives dependencies from more than 20 distinct source components, making it one of the most widely consumed packages in the system. Changes to this package carry the highest ripple-effect risk.
- **context** ↔ **inline** **cycle**: A lower-intensity but structurally notable circular dependency (27/2) is visible in Graph 5, invisible from the matrix alone.

6.4 Conclusion

The dependency analysis of Thymeleaf reveals an architecture that is largely well-structured at the macro level, but exhibits several localized structural risks that warrant targeted attention.

- The most critical finding is the concentration of circular dependencies around the **engine** package, which participates in bidirectional coupling with **util**, **model**, **cache**, **context**, and **templateparser**. These cycles increase the risk of cascading failures and significantly complicate unit testing and independent deployment of individual components.
- The **util** package, while correctly functioning as a shared utility hub, is depended upon by virtually the entire system. This makes it an exceptionally high-impact change surface: any modification to **util** has the potential to affect all consuming packages simultaneously. Strict stability guarantees should be enforced for this package.
- The **templatemode** package presents a similar concern from a fan-in perspective, receiving dependencies from over 20 source components. It should be treated as a stable, interface-like contract within the system.
- On the positive side, leaf packages such as **preprocessor**, **postprocessor**, **messengeresolver**, and **linkbuilder** demonstrate well-isolated, single-responsibility profiles consistent with sound modular design. The **exceptions** package correctly functions as a dependency sink with no outgoing coupling.

Priority actions: Breaking the circular dependencies centered on **engine** — particularly the **engine** ↔ **model** and **engine** ↔ **util** cycles — should be treated as the highest-priority refactoring objective, as resolving these cycles would have the most significant positive impact on the system's long-term maintainability and testability.

7. OVERALL EVALUATION OF CONCRETE ARCHITECTURE

The metric-based analysis in Section 5 and the dependency analysis in Section 6 are largely convergent, together presenting a coherent and predominantly positive picture of Thymeleaf's architecture.

- Both analyses consistently confirm that the system is modular and loosely coupled at the macro level. The CF value of 21% is structurally supported by the dependency graphs, which show that the majority of packages fulfill clearly bounded roles. Packages such as **preprocessor**, **postprocessor**, **messengeresolver**, and **linkbuilder** are identified as well-isolated leaf components by both the matrix and the graphs. The central role of **engine** as the system's architectural hub, and the broad fan-in profile of **templatemode**, are likewise confirmed by both analyses.
- The most notable divergences arise from the directional limitations of the matrix. Read in isolation, the matrix suggested that **util** had extensive outgoing dependencies — a concern

that the graphs corrected by revealing it to be a fan-in hub with minimal outgoing coupling. Similarly, the apparent outgoing dependencies of **exceptions** were confirmed by the graphs to be incoming references, consistent with the correct behavior of an exception package. More critically, the circular dependencies centered on **engine** — including the **engine** ↔ **model** (485/7) and **engine** ↔ **util** (186/35) cycles — are entirely invisible in the metric analysis but emerge as the most significant structural risk in the dependency graphs.

Overall, Thymeleaf demonstrates a solid and well-maintained architecture for an open-source project of its scale. The identified risks are localized rather than systemic, and resolving the circular dependencies around **engine** represents the highest-priority action for further improving the system's long-term maintainability.

8. REFERENCES

- Google Sheet Document for THYMELEAF metrics:

https://docs.google.com/spreadsheets/d/1JAVd9KZiKUixyzlM3ijMzAcNDFSPCLtDCE1E_ac3x3s/edit?usp=sharing

9. ALLOCATION OF RESPONSIBILITIES WITHIN TEAM MEMBERS

Name of Team Member	Description of Responsibility	Allocation Unit
Rukiye Sedef Keskin	All	All

10. APPENDIX

- 10.1. Appendix A - Thymeleaf GQM Tree: Maintainability Analysis
- 10.2. Appendix B - Thymeleaf Dependency Matrix
- 10.3. Appendix C - Thymeleaf Dependency Graphs